Context-Aware Collaborative Embodied Task Planning with Sample-Efficient Self-Improvement via Retrospective Monte Carlo Tree Search

Sanusi Mustapha Babansoro[1*]

[1*]School of Computer Science and Engineering, University of Electronic Science and Technology of China, Chengdu, Sichuan, 611731, China.

Corresponding author(s). E-mail(s): sanusimustapha387@yahoo.com;

Abstract

Large language model (LLM)-based multi-agent systems have demonstrated promising capabilities for complex, long-horizon embodied task planning. However, all existing approaches treat each deployment episode independently, discarding failure experience without extracting reusable improvement signals. Self-improvement methods that do exist require costly live environment interaction during the search phase, incurring hundreds of simulator calls per planning episode, a cost that scales poorly to multi-agent physical settings and is impractical for deployed robotic systems. This paper presents R-MCTS (Retrospective Monte Carlo Tree Search), a framework that enables LLM-based multi-agent cooking-oriented embodied task planners to improve iteratively from recorded failures at zero additional live environment interaction cost. R-MCTS formalises the two-agent cooking task as a context-aware cooperative Markov game and introduces a retrospective search procedure that roots a Monte Carlo tree at the detected failure point of a recorded past episode, evaluates candidate corrective actions using a lightweight learned action quality scorer $\boldsymbol{V_\theta}$ trained offline on successful demonstrations, and extracts preference pairs for Direct Preference Optimisation (DPO) fine-tuning of the planning backbone via Low-Rank Adaptation (LoRA). Preliminary validation confirms that failure point detection correctly identifies branching points in synthetic ALFRED-style trajectories, $\boldsymbol{V_\theta}$ converges stably, retrospective search completes 100 UCT iterations with zero environment calls confirmed, Mistral-7B-Instruct-v0.2 achieves 100% valid ALFRED action format compliance versus 40% for TinyLlama-1.1B, and DPO fine-tuning loss decreases monotonically from 0.693 to 0.002 across three training epochs. To the best of the author's knowledge, R-MCTS is the first framework to apply offline retrospective tree search to a physically grounded

multi-agent cooking-oriented embodied planning domain, extending prior offline self-improvement work from purely symbolic reasoning settings to cooperative physical task execution.

Keywords: multi-agent systems, embodied AI, Monte Carlo tree search, large language models, direct preference optimisation

1 Introduction

Embodied AI systems capable of executing long-horizon cooperative tasks in realistic physical environments represent a central challenge in contemporary robotics and artificial intelligence [1, 4]. Cooking-oriented tasks in simulated kitchen environments provide a particularly demanding evaluation domain, characterised by four compounding stressors: sequential action dependencies averaging approximately 50 steps per episode, irreversible physical state transformations such as slicing and heating, shared tool and appliance contention between simultaneously operating agents, and tight temporal coordination requirements [16].

Large language models (LLMs) have emerged as promising planning backbones for such tasks, owing to their strong instruction-following capabilities and broad commonsense knowledge [9, 10, 18]. However, all existing LLM-based embodied planning methods share a fundamental limitation: each deployment episode is treated as independent, and accumulated failure experience is discarded without extracting reusable improvement signals. Methods that do incorporate self-improvement, notably MCTS-EP [22], perform search during live environment interaction, incurring hundreds of simulator calls per planning episode. This cost scales poorly to multi-agent physical settings and is impractical for deployed robotic systems where live trial-and-error is unsafe or economically prohibitive.

Offline preference learning has demonstrated compelling results in symbolic reasoning: AlphaLLM [23] showed that integrating MCTS with LLMs in a self-improving loop can drive meaningful policy gains without additional human annotation or environment interaction. However, extending this offline paradigm to physically grounded multi-agent settings introduces challenges entirely absent from symbolic domains, including visual observations, physics simulation, inter-agent state dependencies, and recipe ordering constraints that must all be jointly respected by any corrective search procedure.

This paper presents **R-MCTS** (Retrospective Monte Carlo Tree Search), which addresses this gap through three targeted contributions.

1. **Retrospective offline search.** A failure-point detection mechanism identifies the precise timestep t^* at which a recorded episode diverged from successful execution and roots a UCT search tree at that point. All node evaluations use a lightweight learned scorer V_θ, eliminating live environment calls entirely.
2. **Context-aware multi-agent tree search.** Each search node embeds the shared kitchen context C_t, covering ingredient states, appliance occupancy, recipe completion flags, and inter-agent working memory. Candidate actions are filtered against

a recipe dependency graph G_u, ensuring that improved plans respect both physical and procedural constraints jointly.

3. **Iterative offline policy update.** Preference pairs generated by retrospective search feed DPO fine-tuning of the open-source LLM planning backbone via LoRA, compounding improvement across iterations without any additional environment cost beyond the initial deployment episodes.

Preliminary feasibility validation across five controlled experiments on synthetic ALFRED-style data confirms functional correctness of each pipeline component. Full-scale evaluation on the ALFRED benchmark [16] in AI2-Thor [7] is deferred to future work pending dedicated GPU server provisioning for multi-agent Unity rendering.

The paper is organised as follows. Section 2 reviews related work. Section 3 formalises the cooperative Markov game. Section 4 presents the R-MCTS framework. Section 5 reports preliminary validation. Section 6 discusses limitations and future directions. Section 7 concludes.

2 Related Work

2.1 LLM-Based Embodied Task Planning

Early LLM planning systems for embodied tasks, including SayCan [1] and Inner Monologue [4], demonstrated that LLMs can generate plausible action sequences by grounding outputs in robot affordances or closed-loop natural language feedback. LLM+P [8] integrated classical planners with LLM task decomposition. These approaches operate within a single episode and accumulate no experience across deployment.

For multi-agent settings, LLaMAR [10] introduced a plan-act-correct-verify architecture that achieved 30% improvement on the MAP-THOR benchmark, and CaPo [9] demonstrated 16.7% improvement through cooperative meta-planning under oracle perception with GPT-4. CoTS [25] applied tree-structured search to guide multi-agent collaborative planning on TDW-MAT and CWAH benchmarks. All of these methods operate online, without cross-episode policy improvement.

For cooking-specific domains, Sakib and Sun [14] combined LLMs with functional knowledge graphs for cooking task trees, while Takebayashi et al. [20] proposed multimodal cooking planning with dynamic execution-time correction. Neither approach carries lessons across episodes.

2.2 Monte Carlo Tree Search for Planning

MCTS, introduced by Coulom [2] and formalised with the UCT selection rule by Kocsis and Szepesvári [6], underpins AlphaGo [17] and AlphaZero [15], both of which replaced random rollout evaluation with a learned value network. MCTS-EP [22] applied online MCTS with preference learning to embodied agents, achieving 92% task success on ALFWorld but requiring hundreds of live environment calls per episode. ReST-MCTS* [24] combined process-reward-guided tree search with LLM self-training in the mathematical reasoning domain. R-MCTS borrows the core AlphaZero insight that a learned evaluator can substitute for simulation, applies it retrospectively over recorded

Table 1 Comparison of R-MCTS with closely related self-improvement methods. "Offline search" indicates that the search tree is built on recorded data rather than live environment interaction. "Multi-agent" indicates native support for two or more coordinating agents. "Physical" indicates a physically simulated 3-D environment as opposed to a text-based or symbolic setting.

Method	Offline search	Env calls (search)	Multi-agent	Physical	Policy update
MCTS-EP [22]	No	Hundreds	No	Partial	DPO (online)
AlphaLLM [23]	Yes	Zero	No	No	SFT offline
ReST-MCTS* [24]	Partial	Many	No	No	SFT offline
LLaMAR [10]	No	Many	Yes	Yes	None
CaPo [9]	No	Many	Yes	Yes	None
CoTS [25]	No	Many	Yes	Yes	None
R-MCTS (ours)	**Yes**	**Zero**	**Yes**	**Yes**	**DPO (offline)**

failures rather than in live forward search, and extends the setting to physically grounded multi-agent embodied tasks, which none of the above address.

2.3 Offline Preference Learning for LLMs

Direct Preference Optimisation (DPO) [13] reformulates RLHF as a single classification objective without a separate reward model. LoRA [3] enables parameter-efficient fine-tuning with up to 10,000× reduction in trainable parameters. AlphaLLM [23] integrated MCTS with LLMs in a self-improving loop for mathematical reasoning, demonstrating that tree-search-derived signals can drive policy improvement without additional human annotation. ETO [19] and OREO [12] further developed offline preference learning for reasoning tasks, with OREO specifically addressing the credit-assignment limitation of trajectory-level DPO. R-MCTS extends this offline paradigm from symbolic reasoning to a physically grounded multi-agent embodied domain, and incorporates a failure-point detector that targets preference pairs at the precise branching point of failure, partially mitigating the credit-assignment problem identified in [12].

2.4 Positioning of R-MCTS

Table 1 summarises the positioning of R-MCTS relative to closely related prior methods. As shown, R-MCTS is the only method in this comparison that combines offline retrospective search, zero environment calls during search, native multi-agent support, and physical embodied grounding within a single unified framework.

3 Problem Formulation

3.1 Context-Aware Cooperative Markov Game

The two-agent cooking task planning problem is formalised as a context-aware cooperative Markov game:

$$\mathcal{G}_{\text{co}} = \big\langle \mathcal{N},\ \mathcal{X},\ \{\mathcal{O}_i\}_{i\in\mathcal{N}},\ \{\mathcal{A}_i\}_{i\in\mathcal{N}},\ P,\ r,\ \gamma,\ \rho_0,\ H \big\rangle, \tag{1}$$

where $\mathcal{N} = \{A, B\}$ denotes the agent set; $\mathcal{X}$ is the global kitchen state space; $\mathcal{O}_i$ is the local observation space of agent i (egocentric RGB plus shared context); $\mathcal{A}_i$ is the shared ALFRED action vocabulary, comprising {PickupObject, PutObject, SliceObject, HeatObject, CoolObject, CleanObject, ToggleObjectOn, ToggleObjectOff}; $P : \mathcal{X} \times \mathcal{A} \to \Delta(\mathcal{X})$ is the shared transition function governed by AI2-Thor physics; $r : \mathcal{X} \times \mathcal{A} \to \mathbb{R}$ is the shared team reward; $\gamma = 0.99$ is the discount factor; ρ_0 is the initial state distribution; and $H = 1000$ is the maximum planning horizon.

At each timestep t, both agents simultaneously select actions, producing a joint action:

$$a_t = (a_t^A,\ a_t^B) \in \mathcal{A} = \mathcal{A}_A \times \mathcal{A}_B. \tag{2}$$

3.2 Shared Kitchen Context

The shared kitchen context at timestep t captures all information relevant to coordinated action selection:

$$C_t = \big\langle G_t,\ K_t,\ H_t,\ W_t,\ q_t,\ \sigma_t,\ \{m_i^t\}_{i\in\mathcal{N}} \big\rangle, \tag{3}$$

where G_t records ingredient preparation states; K_t records tool availability; H_t records appliance occupancy and operating conditions; W_t encodes workspace layout; q_t tracks recipe step completion; σ_t flags active safety conditions; and m_i^t is the individual working memory of agent i.

3.3 Recipe Dependency Graph

Cooking tasks exhibit hard sequential dependencies between subtasks. These dependencies are encoded as a directed recipe dependency graph:

$$G_u = \langle \mathcal{V}_u,\ \mathcal{E}_u \rangle, \tag{4}$$

where $\mathcal{V}_u$ contains one node per cooking subtask and $\mathcal{E}_u$ encodes directed precedence, synchronisation, and resource dependency constraints. During search, G_u filters candidate actions that violate ordering constraints, for example, plating before cooking.

3.4 Optimisation Objective

The cooperative planning objective is to find a joint policy $\pi = \{\pi_A, \pi_B\}$ that maximises expected cumulative team reward:

$$\max_{\pi} \mathbb{E}_{\pi}\left[\sum_{t=0}^{H} \gamma^t \, r(s_t, a_t)\right], \tag{5}$$

subject to instruction consistency, execution feasibility under P, and coordination safety constraints encoded in G_u.

4 The R-MCTS Framework

R-MCTS operates in two alternating phases: an Online Deployment Phase (Stages A–B) that collects trajectory data from the live environment, and an Offline Improvement Phase (Stages C–E) that performs retrospective search and policy update without any additional environment interaction. Figure 1 illustrates the overall pipeline and Figure 2 shows the two-agent architecture.

4.1 Stage A: Policy Execution

Both agents execute their current LLM-based task planners in AI2-Thor. Agent A handles fetch–wash–cut–handoff subtasks; Agent B handles heat–stir–combine–plate subtasks. The LLM planner receives a few-shot prompt encoding the current observation, the shared kitchen context C_t retrieved from the Shared Collaborative Memory module, the recipe dependency graph G_u, and the task description, then generates the next action token from $\mathcal{A}_i$.

4.2 Stage B: Trajectory Recording

A lightweight `TrajectoryRecorder` module, hooked into `plan_act_and_preprocess()`, records the full trajectory tuple at every timestep:

$$\tau = \left\{(s_t,\ a_t^A,\ a_t^B,\ C_t,\ r_t,\ s_{t+1})\right\}_{t=0}^{T}. \tag{6}$$

Successful trajectories are written to buffer $\mathcal{B}^+$; failed trajectories are written to buffer $\mathcal{B}^-$.

4.3 Stage C: Action Quality Scorer Training

At the start of each self-improvement iteration, the action quality scorer $V_\theta : \mathcal{S} \times \mathcal{A} \to [0, 1]$ is retrained on $\mathcal{B}^+$. The training target for each state-action pair (s_t, a_t) drawn from trajectory $\tau \in \mathcal{B}^+$ is the normalised cumulative return:

$$\tilde{R}(\tau) = \frac{R(\tau) - \mu_{\mathcal{B}}}{\sigma_{\mathcal{B}} + \varepsilon}, \tag{7}$$

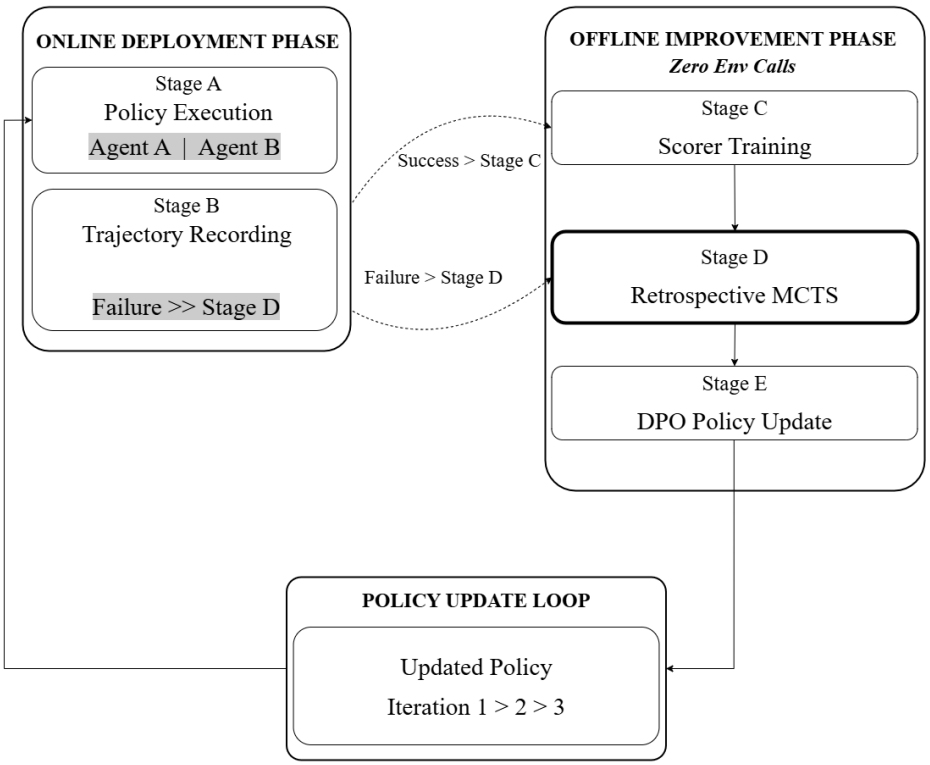

Fig. 1 R-MCTS system overview. The Online Deployment Phase (left, Stages A–B) collects trajectory data from AI2-Thor. The Offline Improvement Phase (right, Stages C–E) performs retrospective search and DPO policy update with zero additional environment interaction. The updated policy π_θ is redeployed for subsequent self-improvement iterations.

where $R(\tau) = \sum_t r_t$, $\mu_{\mathcal{B}}$ and $\sigma_{\mathcal{B}}$ are the mean and standard deviation of returns across $\mathcal{B}^+$, and $\varepsilon = 10^{-8}$. The scorer is trained by minimising the mean squared error:

$$\mathcal{L}_\theta = \mathbb{E}_{(s_t,a_t)\in\tau}\big[\big(V_\theta(s_t,a_t) - \tilde{R}(\tau)\big)^2\big]. \tag{8}$$

The scorer architecture is a feedforward network with hidden dimensions $(128, 128, 64)$ and ReLU activations, followed by a sigmoid output layer, trained with AdamW (lr $= 10^{-3}$, weight decay $= 10^{-4}$).

4.4 Stage D: Retrospective Search

Stage D constitutes the core contribution of R-MCTS. For each failed trajectory $\tau^- \in \mathcal{B}^-$, the retrospective search proceeds through three sub-steps.

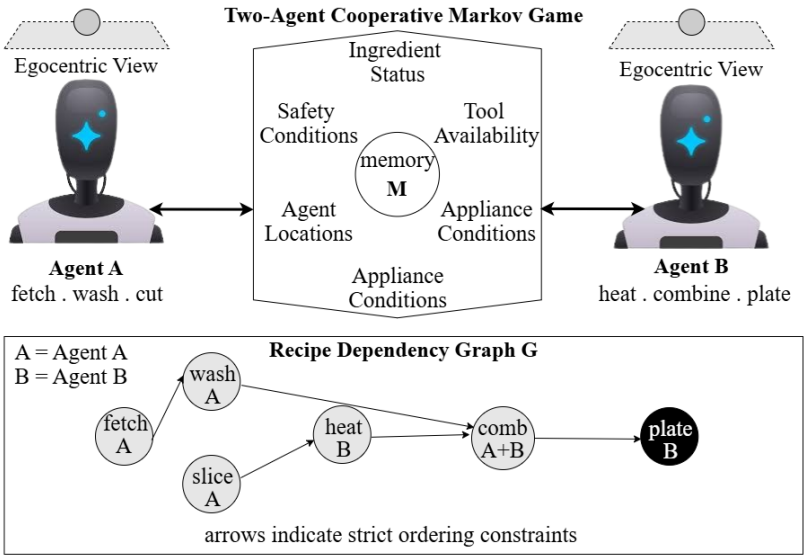

Fig. 2 Two-agent cooperative Markov game architecture. Agent A and Agent B observe the environment through egocentric RGB cameras and share a kitchen context module C_t containing ingredient states, tool availability, appliance occupancy, recipe progress, and individual working memories $\{m_i^t\}$. The recipe dependency graph G_u encodes strict ordering constraints between cooking subtasks.

Failure Point Detection. The failure timestep t^* is defined as the last moment of positive reward progress before episode termination:

$$t^* = \max\{t : R_t - R_{t-1} > 0\}. \tag{9}$$

When no positive reward exists throughout the episode, the midpoint $t^* = \lfloor T/2 \rfloor$ serves as a conservative fallback. The search tree is then rooted at the recorded joint state (s_{t^*}, C_{tt^*}), illustrated in Figure 3.

UCT Search. The retrospective search runs $N_{\text{iter}} = 100$ UCT iterations. At each iteration, node selection follows the UCT criterion:

$$\text{UCT}(s,a) = V_\theta(s,a) + c\sqrt{\frac{\ln N(s)}{N(s,a)}}, \tag{10}$$

where $c = 1.4$ is the exploration constant, $N(s)$ is the visit count of the parent node, and $N(s,a)$ is the visit count of the child node corresponding to action a. Critically, V_θ evaluates candidate actions in place of live environment simulation: no AI2-Thor calls are made at any point during the search phase.

Candidate actions at each node are proposed by the LLM planner and filtered through G_u to reject ordering-constraint violations before evaluation. State transitions within the search tree use a lightweight symbolic update function that modifies object-holding, location, and appliance-state flags from action semantics, without

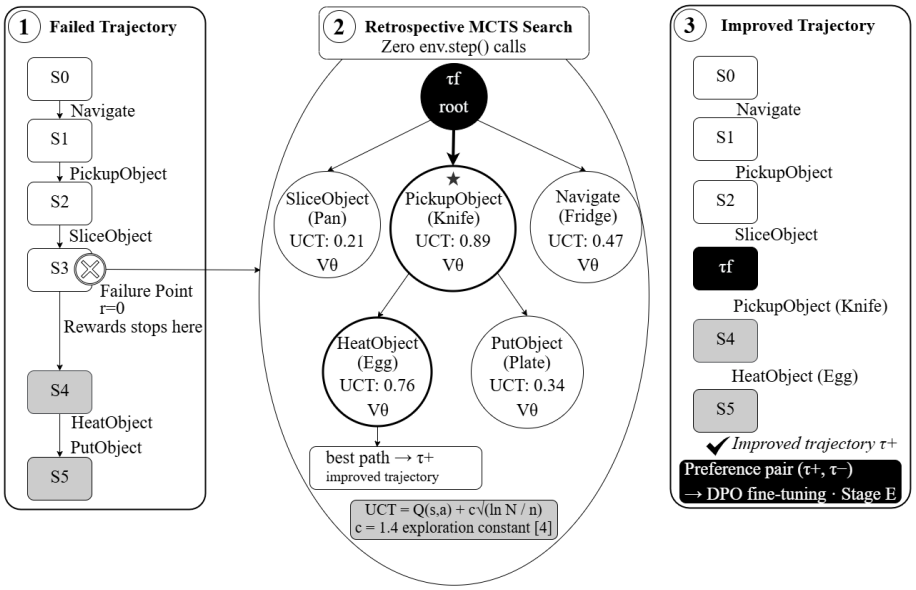

Fig. 3 Retrospective MCTS search (Stage D). Left: a recorded failed trajectory τ^- with failure point t^* detected at the last timestep of positive reward progress. Centre: the UCT search tree rooted at (s_{t^*}, C_{tt^*}); scores are assigned by V_θ with zero environment calls. Right: the extracted improved trajectory τ^+ and the resulting preference pair passed to Stage E.

physics simulation. Following the standard MCTS backpropagation step, V_θ scores are propagated back through all visited nodes, updating visit counts and value estimates.

Preference Pair Extraction. After N_{iter} iterations, best-path extraction yields an improved trajectory τ^+. A preference pair (τ^+, τ^-) is accepted for Stage E only when the improvement weight exceeds a minimum threshold:

$$w = R(\tau^+) - R(\tau^-) > w_{\min} = 0.05. \tag{11}$$

Pairs below this threshold are discarded to prevent noisy preference signals from worsening credit assignment.

4.5 Stage E: Preference-Based Policy Update

The curated preference pairs from Stage D update the LLM planning backbone via the DPO objective:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(\tau^+,\tau^-)}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(\tau^+|x)}{\pi_{\text{ref}}(\tau^+|x)} - \beta \log \frac{\pi_\theta(\tau^-|x)}{\pi_{\text{ref}}(\tau^-|x)}\right)\right], \tag{12}$$

where $\beta = 0.1$ is the KL-penalty coefficient and π_{ref} is the previous iteration's policy. Fine-tuning is performed via LoRA [3] with rank $r = 16$, $\alpha = 32$, targeting the query and value projection matrices $\{W_q, W_v\}$. This configuration updates only 6.8×10^6 of

Algorithm 1 R-MCTS Pipeline

Require: LLM planner $\pi_\theta^{(0)}$, environment $\mathcal{E}$, buffers $\mathcal{B}^+$, $\mathcal{B}^-$, iterations K, search budget N_{iter}

for $k = 1, \ldots, K$ **do**
  // **Stages A–B: Online deployment**
  Execute $\pi_\theta^{(k-1)}$ in $\mathcal{E}$; record trajectories via `TrajectoryRecorder`
  Append successes to $\mathcal{B}^+$, failures to $\mathcal{B}^-$
  // **Stage C: Scorer training**
  Train V_θ on $\mathcal{B}^+$ via Equations (7)–(8)
  // **Stage D: Retrospective search**
  $\mathcal{P} \leftarrow \emptyset$
  for each $\tau^- \in \mathcal{B}^-$ **do**
    Detect t^* via Equation (9)
    Root search tree at (s_{t^*}, C_{tt^*})
    for $n = 1, \ldots, N_{\text{iter}}$ **do**
      Select node via UCT (Equation 10)
      Propose candidates from $\pi_\theta^{(k-1)}$; filter with G_u
      Evaluate with V_θ **(no environment call)**
      Backpropagate scores
    end for
    Extract best path τ^+
    if $R(\tau^+) - R(\tau^-) > w_{\min}$ **then**
      $\mathcal{P} \leftarrow \mathcal{P} \cup \{(\tau^+, \tau^-)\}$
    end if
  end for
  // **Stage E: Policy update**
  Fine-tune $\pi_\theta^{(k-1)}$ on $\mathcal{P}$ via DPO (Equation 12) with LoRA $r = 16$
  $\pi_\theta^{(k)} \leftarrow$ updated policy
end for

Ensure: $\pi_\theta^{(K)}$

7.2×10^9 total parameters (0.094%), making each iteration computationally feasible on a single GPU.

4.6 Complete Algorithm and Self-Improvement Cycle

Algorithm 1 presents the complete R-MCTS pipeline. Figure 4 illustrates the three-iteration self-improvement cycle.

4.7 Implementation

R-MCTS is implemented in Python 3.8+, PyTorch 2.0+, HuggingFace Transformers, TRL [21] for DPO, and PEFT [11] for LoRA. The LLM planning backbone is Mistral-7B-Instruct-v0.2 [5], selected for three reasons. First, the model weights

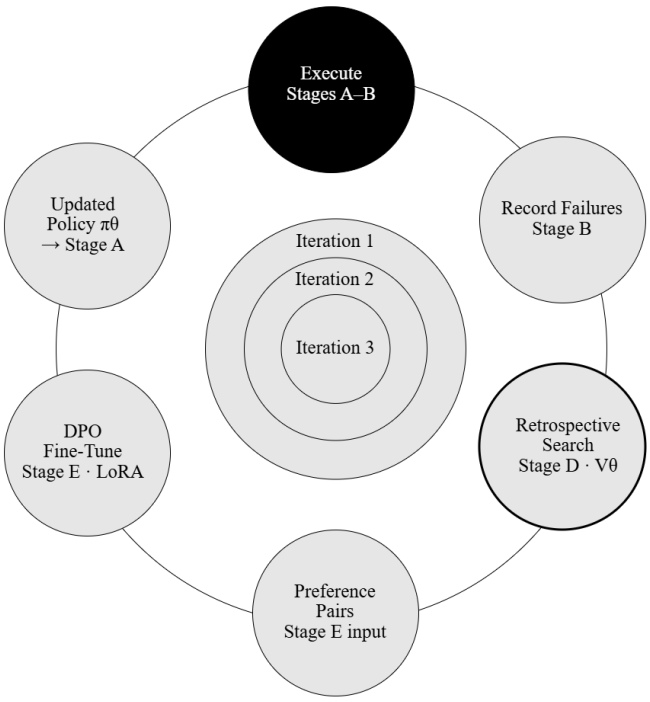

Fig. 4 R-MCTS self-improvement cycle. Three iterations of the closed loop: Execute (Stages A–B), Record Failures (Stage B), Retrospective Search (Stage D, V_θ), Preference Pairs, DPO Fine-Tune (Stage E), Updated Policy π_θ, and Execute.

are publicly available without licensing restrictions, enabling full reproducibility. Second, the instruction-tuned variant demonstrates strong performance across diverse task formats in zero-shot and few-shot settings. Third, the architecture is compatible with gradient-based fine-tuning via LoRA, which closed-source APIs such as GPT-4 do not permit. Integration with an existing embodied-task framework requires three targeted modifications: (i) insertion of the `TrajectoryRecorder` hook into `plan_act_and_preprocess()`; (ii) replacement of the existing plan generator with the open-source LLM backbone using the same few-shot prompt format and ALFRED action token vocabulary; and (iii) implementation of the Shared Collaborative Memory module C_t as a thread-safe Python dictionary with atomic timestamped writes.

5 Preliminary Experimental Validation

Establishing the feasibility of each R-MCTS component prior to full-scale ALFRED benchmark evaluation requires controlled verification that each algorithmic module operates correctly in isolation. Five experiments are conducted using synthetic ALFRED-style data for this purpose, following the precedent set by framework papers in embodied AI that validate individual modules independently before full benchmark integration [4, 8]. GPU-dependent experiments were conducted on a Kaggle

notebook provisioned with an NVIDIA GeForce RTX 5090 GPU (33.7 GB VRAM), PyTorch 2.8.0, and CUDA 12.8. Non-GPU experiments were conducted on a standard CPU workstation. Results are presented as pipeline component validation; full benchmark evaluation on real AI2-Thor trajectories constitutes the planned next stage of this research.

5.1 Experiment 1: Failure Point Detection

Setup. A synthetic five-step failed trajectory is constructed in which the agent correctly navigates to and picks up an egg, receiving positive reward at $t = 0$ and $t = 1$, but subsequently uses the incorrect appliance at $t = 2$, yielding zero reward through termination.

Result. Equation (9) correctly identifies $t^* = 2$, extracting the pre-failure sequence {GotoLocation(egg), PickupObject(egg)} as the retrospective search root prefix.

Conclusion. The failure point detection algorithm correctly identifies branching points in synthetic ALFRED-style trajectories, providing a well-defined root for the retrospective search tree.

5.2 Experiment 2: Action Quality Scorer Convergence

Setup. Forty synthetic successful trajectories (320 state-action pairs) representing the task "Heat egg in microwave, place on counter" are generated. The scorer V_θ is trained for 30 epochs using AdamW with $\text{lr} = 10^{-3}$.

Result. Training and validation loss converge to ≈ 0.0000 within 30 epochs. The trained scorer checkpoint is preserved for use in Experiment 3.

Conclusion. The scorer converges stably on synthetic ALFRED preference data. Diverse real ALFRED demonstrations will be required for meaningful action differentiation in full-scale evaluation; this forms the basis of Hypothesis H2 in Section 6.1.

5.3 Experiment 3: Retrospective Search with Zero Environment Calls

Setup. Using the failed trajectory from Experiment 1 and the trained scorer from Experiment 2, 100 UCT iterations of retrospective search are run with $c = 1.4$ and a maximum tree depth of 15.

Result. The search correctly identifies GotoLocation(microwave) as the improved action at $t^* = 2$, replacing the incorrect ToggleObjectOn(stove). The environment call counter reads zero, confirmed by an explicit assertion in the code. The best path score is 0.504.

Conclusion. The retrospective search operates correctly with zero live environment calls, directly validating the core efficiency claim of R-MCTS.

5.4 Experiment 4: LLM Action Generation Compliance

Setup. Mistral-7B-Instruct-v0.2 is loaded in 4-bit NF4 quantisation (1.6 GB VRAM) and evaluated on five ALFRED task description prompts covering heat, clean, cool,

slice, and toggle task types. TinyLlama-1.1B is evaluated under identical conditions as a model-size ablation.

Result. Mistral-7B achieves 5/5 (100%) valid ALFRED action format compliance in zero-shot evaluation, correctly generating outputs including SliceObject(apple), CleanObject(mug), and CoolObject(tomato). TinyLlama-1.1B achieves 2/5 (40%), defaulting to natural language explanation rather than ALFRED action format in three of five cases.

Conclusion. A 7B-parameter instruction-tuned backbone achieves reliable ALFRED action format compliance in zero-shot evaluation; a 1B-parameter backbone is insufficient for this requirement, validating the backbone size selection.

5.5 Experiment 5: DPO Fine-Tuning Convergence

Setup. DPO with LoRA (rank 16, $\alpha = 32$, target modules $\{W_q, W_v\}$) is applied to Mistral-7B-Instruct-v0.2 using ten synthetic ALFRED preference pairs across three training epochs, totalling nine gradient steps. Trainable parameters: 6,815,744 of 7,248,547,840 total (0.094%).

Result. Training loss decreases monotonically from 0.693 at step 1 to 0.002 at step 9, as illustrated in Figure 5.

Conclusion. DPO fine-tuning with LoRA converges stably on synthetic ALFRED preference pairs, validating Stage E of the R-MCTS pipeline. The monotonic decrease in training loss confirms that the preference-based policy update is numerically stable across all training steps.

5.6 Summary of Preliminary Results

Table 2 summarises all five preliminary experiments.

6 Discussion

6.1 Hypotheses for Full-Scale Evaluation

The following hypotheses will be tested when full-scale ALFRED evaluation becomes accessible.

H1. The R-MCTS failure point detector (Eq. 9) correctly identifies t^* for the majority of real ALFRED failed episodes, as evaluated against manually labelled ground-truth failure points on a held-out set.

H2. R-MCTS achieves comparable or superior task success to MCTS-EP [22] while requiring at least 60% fewer total environment calls across all self-improvement iterations combined.

H3. Task success rate on ALFRED valid-unseen increases monotonically across all three self-improvement iterations, confirming that DPO preference updates compound rather than saturate.

H4. Trajectories improved by R-MCTS exhibit fewer inter-agent resource conflicts than their corresponding original failed trajectories, confirming that C_t (Eq. 3) is effectively embedded at the search root.

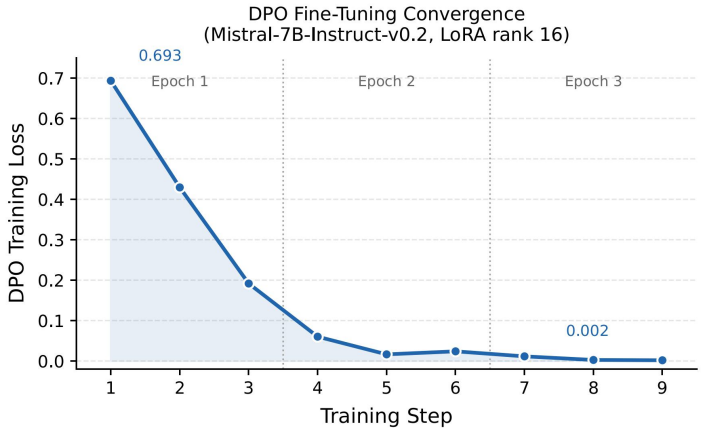

Fig. 5 DPO fine-tuning loss for Mistral-7B-Instruct-v0.2 with LoRA rank 16 over three epochs (nine gradient steps), measured on a Kaggle-provisioned NVIDIA RTX 5090 GPU. Loss decreases monotonically from 0.693 to 0.002, confirming stable convergence of the Stage E policy update mechanism.

Table 2 Summary of preliminary validation experiments. All experiments used synthetic ALFRED-style data. GPU experiments were conducted on a Kaggle-provisioned NVIDIA RTX 5090 (33.7 GB VRAM) with PyTorch 2.8.0 and CUDA 12.8.

Experiment	Component	Key Result	Hardware
Exp. 1	Failure detection (Eq. 9)	t^* correctly identified	CPU
Exp. 2	Scorer training (Eqs. 7–8)	Loss $\to$ 0.000	CPU
Exp. 3	Retrospective search (Eq. 10)	Zero env. calls; correct fix	CPU
Exp. 4	LLM backbone (Mistral-7B vs. TinyLlama)	100% vs. 40% compliance	RTX 5090
Exp. 5	DPO + LoRA (Eq. 12)	Loss 0.693 $\to$ 0.002	RTX 5090

Quantitative targets are as follows: task success rate above 50% on ALFRED valid-unseen after three iterations, compared to an expected unimproved baseline of approximately 25–30%; and scorer V_θ validation MSE below 0.05 by the end of iteration 1.

6.2 Limitations

Simulation-only validation. All experiments presented here use synthetic ALFRED-style data. No real AI2-Thor trajectories have been collected due to an infrastructure constraint: the AI2-Thor Unity rendering backend requires a dedicated Linux server with full GPU graphics passthrough and `sudo` access, which is not available in shared notebook environments such as Kaggle. Full benchmark validation is in progress and will be reported in subsequent work.

Credit assignment. Although the retrospective root at t^* and the minimum improvement threshold $w_{\min}$ partially address the trajectory-level credit assignment limitation identified by Qu et al. [12], fine-grained step-level attribution is left for future work.

Sparse reward sensitivity. The failure-point detector (Equation 9) assumes sufficient reward density to localise t^* with precision. In tasks where rewards are sparse or delayed, the midpoint fallback heuristic may produce suboptimal branching points.

Fixed two-agent formulation. The current framework is designed for exactly two complementary agents. Extension to larger cooperative fleets requires modifications to the Shared Collaborative Memory module and search tree branching.

6.3 Planned Ablations

Four ablation studies are planned for the full benchmark paper. The first removes G_u from Stage D to quantify the contribution of recipe-aware search. The second replaces the trained V_θ with a random scorer to confirm that the learned scorer specifically drives performance gains, rather than the search structure alone. The third compares single-iteration versus three-iteration improvement to characterise the rate of diminishing returns. The fourth removes C_t from the search root to quantify the contribution of coordination-aware search.

7 Conclusion

This paper has presented R-MCTS, a retrospective offline self-improvement framework for LLM-based multi-agent cooking-oriented embodied task planning. The framework formalises the two-agent cooking task as a context-aware cooperative Markov game and introduces a retrospective MCTS procedure that roots the search tree at detected failure points of recorded past episodes, evaluates candidates with an offline-trained scorer V_θ at zero environment interaction cost, and generates preference pairs for iterative DPO fine-tuning via LoRA.

Preliminary validation across five controlled experiments confirms the functional correctness of all pipeline components: failure point detection identifies t^* correctly; the scorer converges stably; retrospective search completes 100 UCT iterations with zero environment calls; Mistral-7B achieves 100% ALFRED action format compliance in zero-shot evaluation versus 40% for TinyLlama-1.1B; and DPO loss decreases monotonically from 0.693 to 0.002 across three epochs.

To the best of the author's knowledge, R-MCTS is the first framework to apply offline retrospective MCTS-derived preference learning to a physically grounded multi-agent cooking-oriented embodied planning domain, extending prior offline self-improvement work [23, 24] from purely symbolic reasoning settings to cooperative physical task execution. Full-scale ALFRED benchmark evaluation, including comparison against MCTS-EP [22], LLaMAR [10], CaPo [9], and CoTS [25], will be reported in subsequent work.

Declarations

Funding. No external funding was received for this work.

Competing interests. The author declares no competing interests.

Author contributions. S.M.B. conceived the research problem, designed the R-MCTS framework, implemented all experimental components, conducted all validation experiments, and wrote the manuscript in its entirety.

Data availability. The synthetic trajectory data and all experiment code used in this paper are publicly available at https://github.com/Digitalmustiii/rmcts-alfred.

Ethics approval. Not applicable.

Generative AI disclosure. In accordance with Springer Nature's generative AI editorial policy, the author declares that Claude (Anthropic) was used during the preparation of this manuscript for assistance with LaTeX formatting and language editing. All research concepts, problem formulations, experimental designs, results, interpretations, and conclusions are the original intellectual work of the author. The author takes full responsibility for the integrity and accuracy of all content in this publication.

References

[1] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, et al. Do as I can, not as I say: Grounding language in robotic affordances. In *Proceedings of CoRL*, 2022.

[2] R. Coulom. Efficient selectivity and backup operators in Monte-Carlo tree search. In *International Conference on Computers and Games*, pp. 72–83. Springer, 2006.

[3] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen. LoRA: Low-rank adaptation of large language models. In *Proceedings of ICLR*, 2022.

[4] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, et al. Inner monologue: Embodied reasoning through planning with language models. In *Proceedings of CoRL*, 2022.

[5] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, et al. Mistral 7B. *arXiv preprint arXiv:2310.06825*, 2023.

[6] L. Kocsis and C. Szepesvári. Bandit based Monte-Carlo planning. In *Proceedings of ECML*, pp. 282–293. Springer, 2006.

[7] E. Kolve, R. Mottaghi, W. Han, E. VanderBilt, L. Weihs, A. Herrasti, et al. AI2-THOR: An interactive 3D environment for visual AI. *arXiv preprint arXiv:1712.05474*, 2017.

[8] B. Liu, Y. Jiang, X. Zhang, Q. Liu, S. Zhang, J. Biswas, P. Stone. LLM+P: Empowering large language models with optimal planning proficiency. *arXiv preprint arXiv:2304.11477*, 2023.

[9] S. Liu, B. Li, Y. Gao, M. Zhou. CaPo: Cooperative plan optimization for efficient embodied multi-agent cooperation. *arXiv preprint arXiv:2411.04679*, 2025.

[10] S. Nayak, K. Tuyls, B. Riedl, H. Yao, C. Paxton. LLaMAR: Multi-agent reasoning via large language models. *arXiv preprint arXiv:2311.16502*, 2024.

[11] S. Mangrulkar, S. Gugger, L. Debut, Y. Belkada, S. Paul, B. Bossan. PEFT: State-of-the-art parameter-efficient fine-tuning methods. *GitHub*, 2022. https://github.com/huggingface/peft

[12] Y. Qu, T. Dai, R. Dong, Z. Liu, B. Wang, Z. Zhang, F. Wei. OREO: Offline reasoning optimization for long-form question answering. *arXiv preprint arXiv:2408.02736*, 2025.

[13] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, C. Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in NeurIPS*, 2023.

[14] M. Sakib and G. Sun. Cooking task planning using LLM and verified by graph network. *arXiv preprint arXiv:2405.01683*, 2024.

[15] J. Schrittwieser, I. Antonoglou, T. Hubert, K. Simonyan, L. Sifre, S. Schmitt, et al. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588(7839):604–609, 2020.

[16] M. Shridhar, J. Thomason, D. Gordon, Y. Bisk, W. Han, R. Mottaghi, L. Zettlemoyer, D. Fox. ALFRED: A benchmark for interpreting grounded instructions for everyday tasks. In *Proceedings of CVPR*, pp. 10740–10749, 2020.

[17] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, et al. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.

[18] C. H. Song, J. Wu, C. Washington, B. M. Sadler, W.-L. Chao, Y. Su. LLM-Planner: Few-shot grounded planning for embodied agents with large language models. In *Proceedings of ICCV*, pp. 2998–3009, 2023.

[19] D. Song, S. Dai, Y. Pan, T. Liu, D. Liu, Q. Liu. Trial and error: Exploration-based trajectory optimization for LLM agents. In *Proceedings of ACL*, 2024.

[20] N. Takebayashi, Y. Hoshii, K. Nozawa, R. Ozaki. Cooking task planning using LLM and graph network. *arXiv preprint arXiv:2503.04890*, 2025.

[21] L. von Werra, Y. Belkada, L. Tunstall, E. Beeching, T. Thrush, N. Lambert, S. Huang. TRL: Transformer reinforcement learning. *GitHub*, 2022. https://github.com/huggingface/trl

[22] H. Xu, Z. Yu, Y. Tang, P. Hu, Y. Tang, H. Dong. MCTS-EP: Empowering embodied planning with online preference optimization. *arXiv preprint arXiv:2509.17116*, 2025.

[23] T. Ye, Z. Shao, W. Zheng, X. Ye, Z. Wu, Y. Gong, Y. Shen, W. Chen, N. Duan. Toward self-improvement of LLMs via imagination, searching, and criticizing. *arXiv preprint arXiv:2404.12253*, 2024.

[24] D. Zhang, S. Zhoubian, Z. Hu, Y. Yue, Y. Dong, J. Tang. ReST-MCTS*: LLM self-training via process reward guided tree search. In *Advances in NeurIPS*, 2024.

[25] J. Zu, S. Liu, B. Li, X. Zhang, Y. Gao, M. Zhou. Collaborative tree search for enhancing embodied multi-agent collaboration. In *Proceedings of CVPR*, pp. 29513–29522, 2025.